

Coroutines – sleep_for

In this paper, I will demonstrate the implementation behind the simple `co_await sleep_for()` call, revealing all the subtle details one should be aware of, the hidden complexity behind the naïve interface, as possible explanation why the coroutines are still, even 6 years after adoption in all major vendor compilers – enigmatic feature that didn't get attention they really deserve.

Introduction

Coroutines are not a novelty, and they are definitely not a C++-specific language feature. Many other languages—including C#, Kotlin, and JavaScript, to name just a few—also support coroutines, with the same underlying meaning: they represent an execution unit (a task) rather than an execution context. Before coroutines, in the traditional thread-centric approach to concurrency, the execution unit was the thread function. As a result, the execution unit and the execution context were effectively the same.

The coroutines we know in C++ are standardized by the group led by the recently passed away **Mr. Gor Nishanov (1971-2026)**, in form of the `<coroutine>` header file – with rudimental support. They are part of all major families of compilers, but there is no standardized coroutine framework. There are only third-party libraries.

I've recently had an online talk where I've demonstrated the coroutines in action, using my socket library as application field.

<Link>: <https://www.youtube.com/watch?v=s93itDBRxPc>

The message to audience, which I've tried to broadcast, is that even if we have the coroutines framework, employing the coroutines as a direct replacement for the existing code is usually an antipattern. The fact is – the entire previous solution – a classical implementation for the receiver component, as the common for both socket endpoints (Server and Client), had to be redesigned to be coroutine-friendly, especially since coroutines are like the Borgs from *Star Trek* – they turn everything they touch, their surrounding environment – into coroutines (Figure-1).

Switching from threads to coroutines and structured concurrency, the coroutine framework needs to redefine the synchronization primitives as well. The std synchronization primitives are expensive OS concepts used for synchronizing the threads, as execution context that is at the same time execution unit. With coroutines in place, we have multiple execution units that may be hosted by the same execution context – the thread (CPU/GPU single-thread, pool of threads, etc.). The same is for `std::this_thread::sleep_for()`, as the way to suspend – block the current thread for a given amount of time. What we want is to reimplement the same mechanism – but in the scope of a single coroutine, suspending the single coroutine for a given time interval, while allowing another coroutine hosted by the same thread to advance – to make a progress.

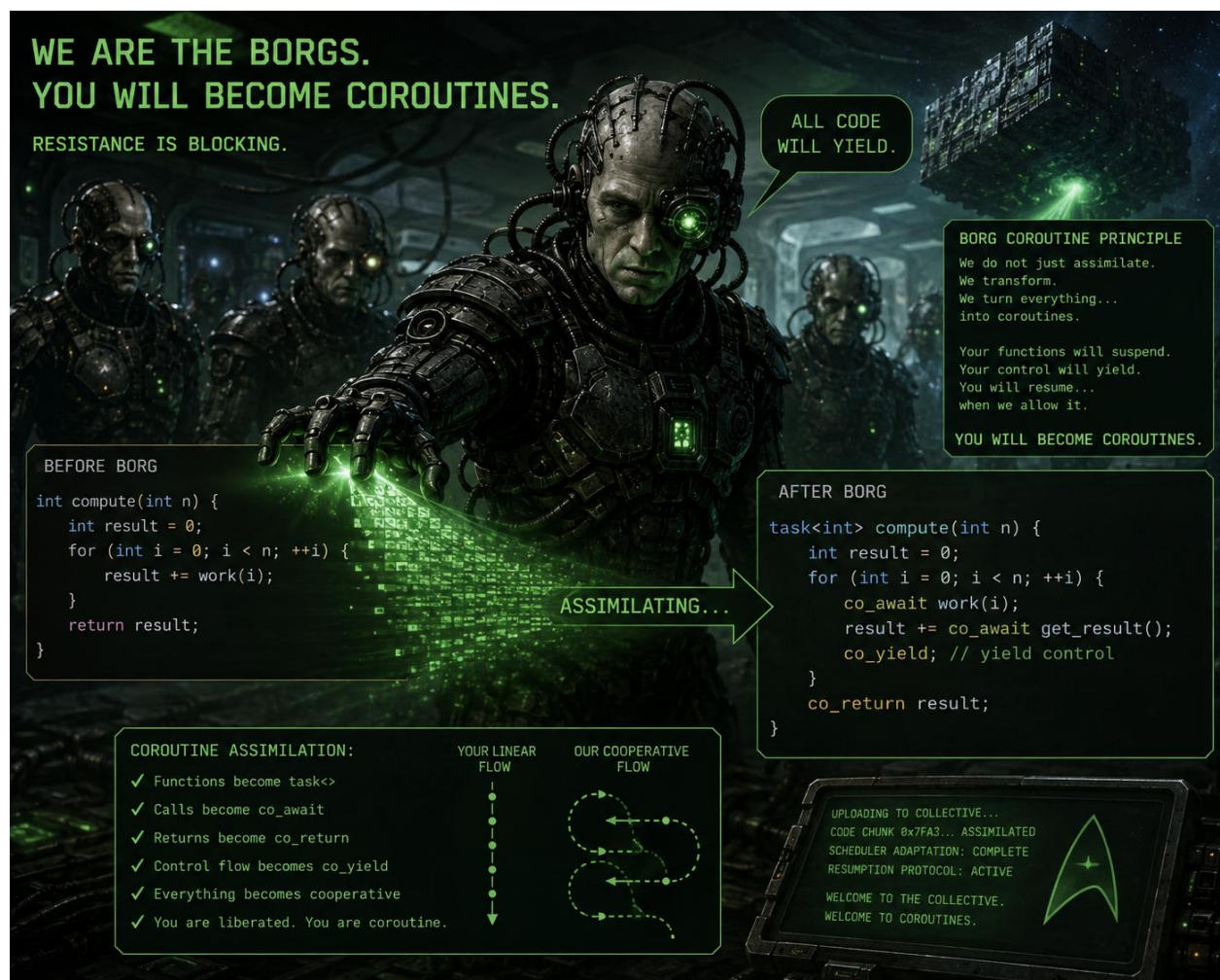


Figure-1. Coroutines as the Borg: A Star Trek Analogy (AI generated image)

Asynchronous sleep_for

The goal that we try to achieve here – is to block a single coroutine, rather than entire thread, for a given amount of time. Eventually, we want to be able to call inside the coroutine

```
co_await details::sleep_for(10ms);
```

What becomes obvious is – we need to implement a matching **awaitable interface** on which we would be able to suspend the caller for the given amount of time.

Assuming that this interface inherits from the `std::suspend_always`

```
template <typename Duration>
struct sleep_for_awaitable final : std::suspend_always
{
    private:
```

Coroutines sleep_for – Damir Ljubic

```

Duration timeout_;
utils::coro::TaskResumeContext* context_{nullptr};

public:
    template <typename D>
        requires std::convertible_to<D, Duration>
    explicit sleep_for_awaitable(D&& timeout,
        utils::coro::TaskResumeContext* context) noexcept
        : timeout_{std::forward<Duration>(timeout)}
        , context_{context}
    {
    }

```

skipping for time being the implementation itself, we can write the helper method

```

template <typename Duration>
[[nodiscard]] auto sleep_for(Duration&& timeout,
    utils::coro::TaskResumeContext* context = nullptr) noexcept
{
    return sleep_for_awaitable<Duration>{
        std::forward<Duration>(timeout), context};
}

```

that returns our awaitable interface implementation.

utils::coro is namespace from my small coroutine framework.

The **TaskResumeContext** is the way to specify explicitly the context on which the suspended caller should be resumed – after timeout being expired.

Implementation

What remains is the actual implementation of the suspension point – the **await_suspend()** customization point of the awaitable interface. This is where all magic happens, where we need to block the caller, for the given amount of time.

As mentioned in the introduction – we can't use the `std::this_thread::sleep_for()` since this would block the entire thread. As matter of fact, we can't use any synchronous wait call here. We need a way to suspend the caller, and employ some timer that will, on timeout expiration – resume the caller: either in the timer context, or in the explicitly specified one.

Sounds simple, right?

I guess it's quite obvious, that we can't just delegate this as a job to a background (detached) thread, since this would be too expensive solution – having for each `co_await sleep_for()` call the thread being spawned: that defeats the original idea of having coroutines at the first place. We need to implement this more wisely, having a single timer that will be shared between the coroutines – in the same manner as the Scheduler, or any common component. The coroutines are all about utilizing the resources.

```
void await_suspend(std::coroutine_handle<> caller) noexcept
{
    using namespace std::chrono;
    timer_.schedule(caller, steady_clock::now() + timeout_, context_);
}
```

We have the Timer component `timer_` on which we schedule a new “sleep for” task, as a deadline after which expiration, the caller will be resumed in a given context.

Let’s see, how we can implement the timer that will be shared between the coroutines. It would be basically a background **thread** with a **priority queue**. We will enqueue the waiting jobs along the resuming context, and the priority queue will take care that the jobs are sorted in ascending order – so that the job with nearest deadline is bubbled up on the top of the queue (min-heap).

The waiting job itself can be modeled as

```
struct entry final
{
    std::chrono::time_point<std::chrono::steady_clock> deadline_;
    std::coroutine_handle<> caller_;
    utils::coro::TaskResumeContext* context_;

    bool operator<(const entry& other) const
    { return deadline_ > other.deadline_; }
};
```

We have our deadline to sleep for, the caller that will be resumed afterwards and optionally – the context where the caller should be resumed.

The **operator < ()** is for the reverting the default priority queue sorting logic (max-heap).

Basically, we want as comparator to use `std::greater<>`: to check two subsequent entries and swap them if the $a > b$ – so that we “push” the nearer deadline up in the list.

Let’s see how we can implement the logic around the priority queue, hosted by the background thread. We need to drain the queue and check for the deadlines being hit

```
void drain_due(std::unique_lock<utils::monitor::v1::Monitor<>::lock_t>&& lock)
{
    // Collect all currently due entries while the lock is held.
    due_.clear();
    for (;;)
    {
        if (queue_.empty()) break; // No more tasks to execute.
        auto entry = queue_.top();
        if (entry.deadline_ <= std::chrono::steady_clock::now())
        {
            queue_.pop();
            due_.push_back(entry);
        }
        else
            break; // No more tasks that are due yet.
    }
}
```

```

}

lock.unlock();
for (auto& entry : due_)
{
    if (entry.context_) { entry.context_>resume_task(entry.caller_); }
    else // execution context is the current (timer) thread (!)
    {
        entry.caller_.resume();
    }
}
};

```

The Monitor is custom implementation of the **Monitor Object** concurrency pattern. Basically, it's the way to synchronize the access to the monitored – guarded object and its methods in multithreaded environment, or, as in this case – just to provide the synchronization mechanism using the (mutex, condition_variable) pair.

Worth mentioning, the caller must be resumed when the lock is released, to prevent deadlock – when there is a new entry to be enqueued.

Here is the entire logic, hosted within the thread function run()

```

void run()
{
    using namespace std::chrono;
    using namespace std::chrono_literals;

    auto drain =
        [this] (std::unique_lock<utils::monitor::v1::Monitor<>::lock_t>&& lock)
        { drain_due(std::move(lock)); };

    std::uint64_t currCounter{0};
    for (;;)
    {
        std::optional<entry> entry{};
        monitor_.stopped_or_wait(
            [this] { return not queue_.empty(); },
            [this, &entry, &currCounter]
            {
                entry = queue_.top();
                currCounter = counter_;
            }
        );
        if (monitor_.is_stopped()) break;

        if (entry)
        {
            const auto deadline = (*entry).deadline_;
            const auto waitFor = deadline - steady_clock::now();

            const auto wakeup = [&, this, currCounter] // wakeup predicate
            {
                return queue_.empty() or
                    queue_.top().deadline_ <= steady_clock::now() or

```

```

currCounter != counter_;
};

monitor_.wait_for(std::max(waitFor, 0ns), wakeup, drain);
}
}
}

```

The steps are:

1. We wait until the queue is empty and stop is not signaled
2. Assuming that there is an entry to process
3. We wait on condition_variable until the deadline is hit, or wakeup predicate is met – whatever comes first

The reason why we have **counter** in the predicate is that we can have the single entry being processed (deadline_1 = 45ms) and there is a new entry (deadline_2 = 10ms) which will land on top of the queue, but without satisfying yet the “deadline being expired” condition – it will not be taken into account as a new deadline candidate to wait for: the condition variable continues to wait on the same deadline.

When I run the unit-tests on my Windows machine, surprisingly I’m getting following results

```

..\tests\unittests\build\exe\main\debug\windows\x86-64> .\unittests.exe
[sleep_for]
Filters: [sleep_for]

sleep_for(50 ms) actually took 58 ms
two sequential sleeps (20 ms + 30 ms) took 61 ms in total
short sleep_for(20 ms) took 30 ms
long sleep_for(300 ms) took 313 ms
sleep_for(0 ms) took 0 ms
sleep_for(0 ms) #1 took 0 ms, #2 took 0 ms
sleep_for(0 ms) (queued behind a 300 ms sleep) took 0 ms
long sleep_for(300 ms) took 313 ms
=====
All tests passed (17 assertions in 6 test cases)

```

The `std::condition_variable.wait_for()` seems to have a “wakeup resolution” that is on Windows around 15ms. To fix that, we need a “three-way precision” in place. That is to say

1. We check the remaining time to the first deadline, whether it is bigger than the condition variable slack. In case it is, we will first use the **std::condition_variable::wait_for()**. On the timeout expiration without deadline being signaled, we will repeat it – recalculate the deadline, as long as the remaining time can be safely used with condition_variable precision.
2. Otherwise, in case that remaining time is less than cv slack – we fall back to the **sleep_until()** precision, around the system monotonic clock.
3. Finally, if we are in the range of the sleep granularity, we perform – for the remaining, smallest fraction of time, the busy-loop – yielding control to another thread

phase	primitive	interruptible	granularity	bounded by
1	condition_variable	yes	~15.6 ms / ~100 μ s	—
2	clock_nanosleep(ABSTIME)	no	~50–200 μ s	CV Slack
3	spin on steady_clock	no	~100 ns	Spin Slack

Table-1. The “three-way precision” to overcome the system limitations

We can tune the previous implementation, applying the “three-way precision” as

```
void run()
{
    using namespace std::chrono;
    using namespace std::chrono_literals;

    auto drain =
        [this](std::unique_lock<utils::monitor::v1::Monitor<>::lock_t>&& lock)
        { drain_due(std::move(lock)); };

    const auto granularity = precise::sleep_granularity();
    const auto cvSlack = std::max<nanoseconds>(cv_slack_, granularity);
    const auto spinSlack = std::max<nanoseconds>(spin_slack_, granularity);

    std::uint64_t currCounter{0};
    for (;;)
    {
        std::optional<entry> entry{};
        monitor_.stopped_or_wait(
            [this] { return not queue_.empty(); },
            [this, &entry, &currCounter]
            {
                entry = queue_.top();
                currCounter = counter_;
            }
        );
        if (monitor_.is_stopped()) break;

        if (entry)
        {
            const auto deadline = (*entry).deadline_;

            const auto wakeup = [&, this, currCounter]
            {
                return queue_.empty() or
                    queue_.top().deadline_ <= steady_clock::now() or
                    currCounter != counter_;
            };

            // Check within the wakeup resolution for condition_variable wait
            const auto remaining = deadline - steady_clock::now();
            if (remaining > cvSlack)
            {

```



```

        monitor_.wait_for(remaining - cvSlack, wakeup, drain);
        continue; // re-evaluate: the queue may have changed
    }

    // Phase 2 - precise, blocking, not interruptible, bounded by
    // cvSlack.
    if (remaining > spinSlack)
    { precise::sleep_until(deadline - spinSlack); }

    // Phase 3 - final approach, bounded by spinSlack.
    if constexpr (spin_slack_ > nanoseconds::zero())
    { precise::spin_until(deadline); }

    // The predicate is true by now - drains the due entries
    monitor_.guard(drain);
}
} // for(;;)
}

```

And now, the results are as expected

```

..\tests\unittests\build\exe\main\debug\windows\x86-64> .\unittests.exe
[sleep_for]
Filters: [sleep_for]

sleep_for(50 ms) actually took 50 ms
two sequential sleeps (20 ms + 30 ms) took 50 ms in total
short sleep_for(20 ms) took 20 ms
long sleep_for(300 ms) took 300 ms
sleep_for(0 ms) took 0 ms
sleep_for(0 ms) #1 took 0 ms, #2 took 0 ms
sleep_for(0 ms) (queued behind a 300 ms sleep) took 0 ms
long sleep_for(300 ms) took 300 ms
=====
All tests passed (17 assertions in 6 test cases)

```

Conclusion

The motivation for writing this article is bitter true: the coroutines are not meant for us – ordinary SW developer. They are primary meant for the **library developer**, which also explains the rudimentary language support for the coroutines, with two mandatory interfaces (`promise_type` and `awaitable`). Everything else needs to be built around it – the entire framework.

Even simple – toy example, to have

```
co_await sleep_for(timeout);
```

requires a lot of code – the most functionality in this implementation is hidden behind the utility classes and functions, reusing the existing coroutine framework of mine.

Also, the proper using the coroutines in the code is not trivial at all, especially if you are on the mission to refactor the legacy code – for the reasons already explained. To repeat, just replacing

Coroutines `sleep_for` – Damir Ljubic

the existing code with coroutines is considered as antipattern. Using coroutines efficiently usually means redesigning the code to be coroutine-friendly. That means better utilization of resources (threads) by decoupling the execution context from the execution unit.

The idea was not to discourage you to use the coroutines (on the contrary), just to be aware how challenging it can be.

And we, engineers, love challenges – don't we?